

Gadolinium Concentration Monitor Manual

Version 1.1

Keshav Anand

email will go here

<https://github.com/uno289/GdConcAnalyzer>

July 13, 2026

Contents

1	Introduction	2
2	Dashboard Overview	3
2.1	Overview	3
2.2	Startup and Shutdown	6
2.2.1	Startup	6
2.2.2	Shutdown	6
3	System Structure	7
3.1	File Structure	8
3.2	Code Structure	9
3.2.1	main	9
3.2.2	FileImport	10
3.2.3	SignalProcessor	10
3.2.4	ExponentialFit	10
3.2.5	Calibration	10
3.2.6	DataLogging	11
3.2.7	Grapher	11
3.2.8	eventlogger	11
3.2.9	app	12
4	Common Bugs	13
4.1	Error Codes	13
4.2	Failure Modes	15
4.2.1	Known Limitations	16
5	Maintenance	17
5.1	Best Practices	17
5.2	Adjusting Parameters	18
5.3	Checklists	19
5.3.1	WaveDump2 Configuration	19
5.3.2	Startup	20
5.3.3	Shutdown	20

1 Introduction

This document serves as a technical guide, code documentation, and common bug-fixing manual for the Gadolinium Concentration Analyzer and its associated functions and website. It will be split into several major sections, as follows:

2. Overview

- This section will explain the web dashboard, its various functions, and the information that can be gathered from it. We will also look at proper startup and shutdown procedure.

3. System Structure

- This section will discuss the function and structure of each of the files within the Analysis pipeline and the webpage.

4. Common Bugs

- This section will enumerate and discuss several common bugs and their fixes, along with an explanation of the known failure modes of the Analysis pipeline.

5. Maintenance

- This section will list best practices to maintain the system over time, and has checklists for startup and shutdown.

These sections should provide a comprehensive outline of the analyzer, the dashboard, and their known strengths and failures. In normal use, the only sections necessary to read are [section 2](#) and [section 5](#).

If an unexpected error occurs, the document should cover most general issues ([section 3](#), [section 4](#)). If the issues are not listed below, please feel free to contact the author at [email] to discuss solutions. The latest source code, installation requirements, and release history are maintained in [the project's GitHub repository](#).

2 Dashboard Overview

2.1 Overview

The dashboard is split into three major sections. I will describe them generally, then break them down further. The three sections are:

1. Quick status

- The four boxes show the general status of the scripts. Four green indicators means everything is working as normal; if any turn yellow or red, then the script has failed somewhere.

2. Data visualization

- All graphing lives here. The main graphs shown are the daily concentration, which sits in the top. Shown below it by default is the averaged single trace, with hourly and minutely graphs available as well.

3. Logs

- The event log lives in this section and is enabled by default. There is also a permanent error log and downloads of the datasets.

Quick status: The quick status boxes represent the four major checkpoints of the code. First, the file must be created by WaveDump; then it must be read, analyzed, and sent to the website. Moreover, the concentration shouldn't vary drastically from the prior runs. This order goes from left to right with the status boxes as well. In normal operation, they will stay green, but if something fails, the user will most likely see it here first.

The boxes themselves do not offer particularly detailed information, and are designed to be used primarily as a "health bar" for the program. A shift worker simply needs to look at the website and make sure everything is green. In the event that there is a failure, this manual should cover the fix.

Data visualization: There are four graphs available to the user in this section. The daily visualization is the main graph, and contains the average daily concentration for each day in the past year. In the three-tab section just below it, the default graph shows the single trace that goes into each analysis run, with the exponential fit applied to it. Also available are a minutely graph that

shows the past day's worth of data, and an hourly graph that shows the past week's worth of data.

Logs: In the log section, one can view a rolling cache of the 15 latest actions the scripts have taken, which are split into time, source, severity level, and message sections. In normal operation, everything should generally show as INFO, with WARNING appearing every once in a while if a file is in use. If an error pops up, one can see the cause in the Event Log. If it is a Python exception, the Error Log will keep a permanent log. In the Download tab, one can download the global minutely, hourly, and daily data sets.



2.2 Startup and Shutdown

This section will serve as a detailed outline of the startup and shutdown procedures. In Section 5, I will include a start/stop checklist.

2.2.1 Startup

To properly start the analysis program and dashboard from a full shutdown, first start WaveDump2. Follow the configuration instructions listed in [subsection 5.3.1](#) to make sure WD2 is configured correctly, then start it up WITHOUT saving files to make sure it works as intended.

Next, to start main.py, open a Terminal window. The terminal should start in GdConcAnalyzer's .venv, but if it does not, start Python from the .venv inside of the GdConc folder. Next, run the command `python main.py` from within the Data Analysis folder, and the program should start up (it will tell you it did not read any files, this is intended behavior). While the \Data\New folder does not need to be empty, I recommend clearing all old files from it.

Following main.py, one can start app.py. To do so, navigate to the same Data Analysis folder as we did for main.py in a new terminal window, then run the command `python app.py` to start up the dashboard.

Finally, enable WaveDump2's file saving, and the program should begin taking data as soon as its wait cycle has finished.

2.2.2 Shutdown

This section does not really have to be done in order, but there are a couple considerations to keep in mind when shutting down the programs. Both the analysis script and the dashboard can be shut down by just closing the terminals (or just doing Ctrl-C in the window with no text selected), and WaveDump2 can just be switched off or closed. Keep in mind that closing WaveDump2 can require reconfiguring it after reopening it, and make sure to clear whatever temp file WaveDump2 had in \Data\New.

3 System Structure

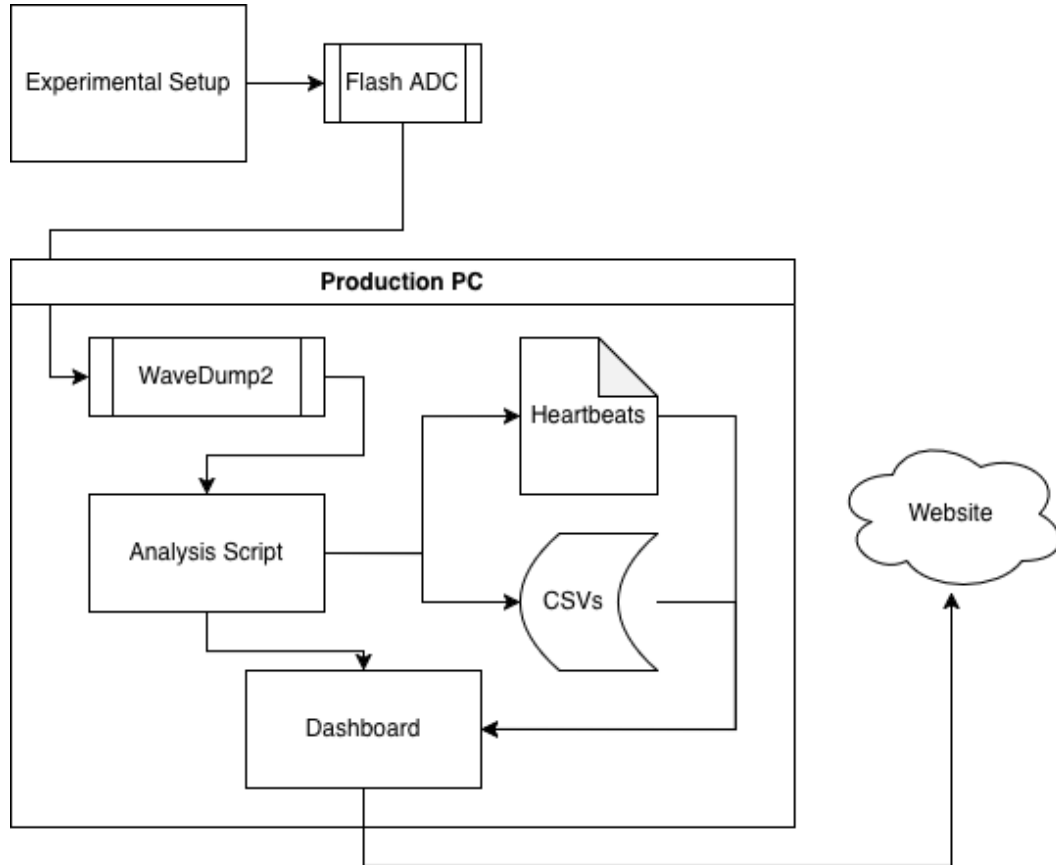


Figure 2: Flow chart of the GdConcAnalysis pipeline.

This section will serve as a rough overview of how the file and code structures work in the GdConcAnalysis project. This section is only necessary to read if the user comes across Python exceptions or other major failures of the code, and none of the code experts are available to fix the issues.

As seen in [Figure 2](#), there is a fairly linear flow through the experimental setup. The analysis script writes to several CSVs, and to some "heartbeat" files that tell the dashboard script that it is active. The dashboard then reads the CSVs to live-generate graphs, and reads the heartbeat files to update the status trackers.

3.1 File Structure

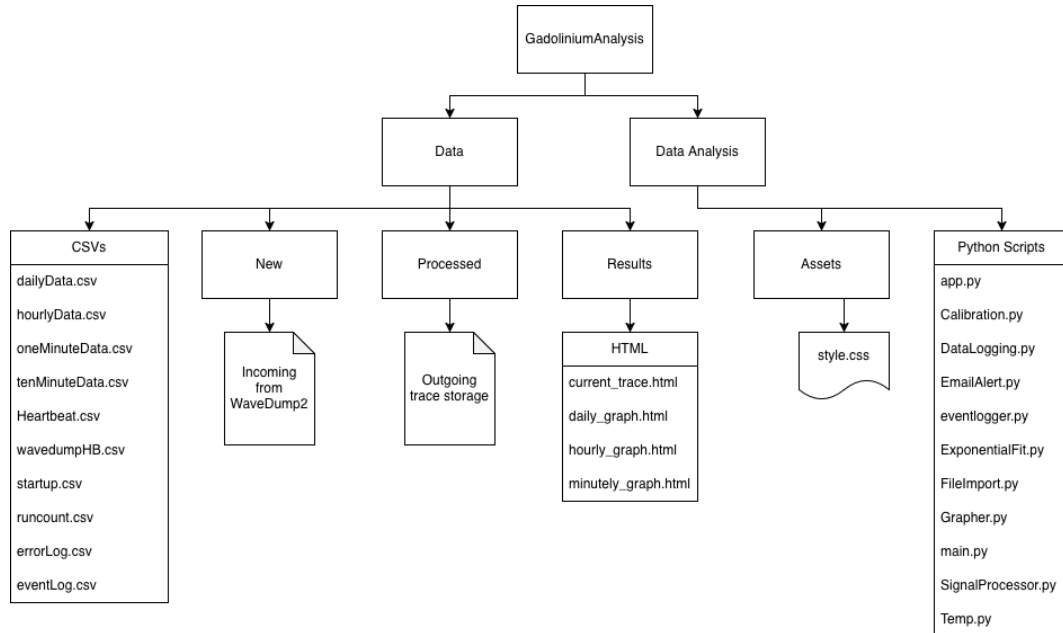


Figure 3: File structure of the GdConcAnalyzer project.

The file structure is not especially complex for the GdConcAnalysis project. All necessary files can be found within `\GadoliniumAnalysis\`, but the folders not immediately necessary for the visual aid (such as the `\.venv\`) have been omitted.

The file structure is generally split into two sections: Data input and storage (`\Data\`), and programming (`\Data Analysis\`). During startup, the only section that needs checks is the storage side.

Data should be set in WaveDump2 to save into the `\Data\New` folder. Once a file of sufficient size enters the folder, it is first checked to make sure it is not still being written into, and then is read and discarded upon completion of the analysis. If an analysis fails, the file will be moved across into `\Data\Processed`, where the user can examine the problematic file.

Long-term data storage happens in several datasets which are made accessible to the user. A minutely dataset, hourly dataset, and daily dataset are saved as CSVs which appear under `\Data\` and are also available on the website. Also within the same directory are the error and event logs, and all the system status indicators.

There are several deprecated files within the structure which were not removable prior to the first iteration of this monitor. None should cause any problems to the user, but they are listed here for completeness: tenMinuteData.csv, daily_graph.html, hourly_graph.html, and minutely_graph.html.

3.2 Code Structure

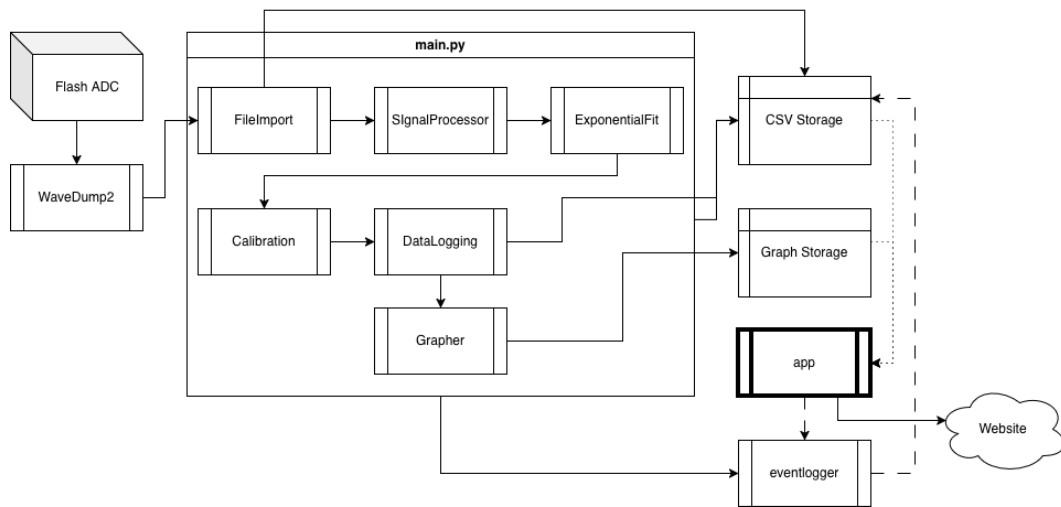


Figure 4: Code structure diagram. Dotted lines are read-only operations, dashed lines are conditional operations.

In this section, we will take a high-level view of each step through the analysis and display pipeline. For a more granular look at the code, please visit the [GdConcAnalyzer GitHub repository](#).

3.2.1 main

Purpose Acts as the primary controller for the analysis pipeline. It monitors for new files, coordinates all analysis modules, and writes the output files consumed by the dashboard.

Inputs WaveDump2 ASCII files stored in Data/New.

Outputs CSV datasets, heartbeat CSVs, event log CSV, error log CSV, and HTML graphs.

Notes Start this script prior to `app.py`. This script runs 24/7 if not stopped. Directly controls every file in the analysis pipeline except for the dashboard (`app.py`). Shares control of event logging with the dashboard.

3.2.2 FileImport

Purpose Scans for new files in `Data/New`, reads files into memory, and moves used files away from the primary folder.

Inputs WaveDump2 ASCII files stored in `Data/New`.

Outputs A list of all N traces, which are each individually their own lists.

Notes Most errors will occur somewhere in this file, usually because the trace count is incorrect.

3.2.3 SignalProcessor

Purpose Aligns all 600 pulses, then averages them into one representative pulse. Recalibrates pulse to remove base voltage.

Inputs List of all 600 traces (each their own list) from `FileImport`.

Outputs Averaged, zeroed, aligned trace (as a list) with N samples.

Notes The pulse alignment target needs to be adjusted depending on the sampling configuration.

3.2.4 ExponentialFit

Purpose Attempts to fit an exponential curve to the 600-trace average.

Inputs Single list of the minute-long average.

Outputs Exponential fit coefficients (V_0, τ) and error bounds.

Notes If the exponential fit starts to fail, check the starting coefficients in this file.

3.2.5 Calibration

Purpose Multiplies $V_0 \times \tau$ with the calibration factor to calculate concentration + error.

Inputs V_0, τ , error values.

Outputs List: [time, concentration, error].

Notes The calibration must be adjusted every time the experimental setup is altered.

3.2.6 DataLogging

Purpose Responsible for all temporary and permanent data storage. Builds temporary caches in RAM for graphing, and permanent CSVs. Also handles Python error logging.

Inputs Individual [time, concentration, error] lists, error log Python exceptions.

Outputs CSVs with permanent storage, error log outputs.

Notes Temporary caches will *not* repopulate after a restart. This is intended behavior, since the system is designed to run (nearly) permanently. The graphs will reload from memory, so the only effect the user will see is the data slightly offset from the prior data.

3.2.7 Grapher

Purpose Graphs the data using Plotly; creates both an offline copy (`\Results\`) and a copy for the dashboard.

Inputs Dataframe containing timestamps, concentration, and error values.

Outputs HTML graphs of 600-averaged trace, minutely, hourly, and daily data.

Notes If this file is updated, restart BOTH scripts, as they both require Grapher.

3.2.8 eventlogger

Purpose Creates the rolling event log shown on the dashboard.

Inputs Individual ["Timestamp","Source","Level","Message"] lists.

Outputs CSV containing 15 most recent events.

Notes To change how many events are shown, change the `max_events` variable. If this script is edited, restart BOTH main and app.

3.2.9 app

Purpose Serves the dashboard using Plotly Dash and periodically updates all displayed data.

Inputs All data and heartbeat CSVs, single-trace HTML.

Outputs Dashboard website, emails upon errors.

Notes Runs 24/7. Uses a Flask server (hosted by Waitress in production) to serve the dashboard over the local network.

4 Common Bugs

4.1 Error Codes

STP001 Startup successful. All sections are operating as intended.

STP002 Startup unsuccessful. Analyzer code not running.

ANX001 Concentration changed by {var}. Check the analysis script.

ANX002 Latest concentration was {var}. Check the analysis script.

ANX003 Unable to process the latest file ({var}). The file was moved to the Processed directory.

HBX001 Heartbeat not detected. Check the analysis script.

FIM001 Imported file had an unexpected length. Check the WaveDump trace count against FileImport.

FIM002 Analyzer stopped. Last file received {var} s ago. Check FileImport.

WDP001 Analyzer stopped. Last file received {var} s ago. Check WaveDump.

WDP002 Critical error. Last file received {var} s ago. Check both WaveDump and FileImport.

In this section, I'll describe some common errors and the ways to fix them. First, we will look at the errors that have codes, and then we will take a look at failure modes of the program. Please note that this is not an exhaustive list - these are the ones I have personally found while creating the program. More may exist, and if they occur, please contact me for assistance.

STP001 is not an error code, but rather a successful startup. All sections of the program are working as intended.

STP002 tells us that main() is not running. The fix is simply to adhere to [startup order](#) - start main(), then start app().

ANX001 is an analysis error, and tells us that there was a large change in the analysis value. Take a look at the recent value, and check the corresponding file if needed.

ANX002 is a specific error, and comes up when the concentration value reads negative. If this happens, there is a major error, as the code should never return negative code. Like ANX001, look at the recent value, and check the corresponding file to see if there was an error.

ANX003 occurs when the file was, for some reason, unable to pass through the file processor and successfully produce code. Take a look at the file that was input, and see if there was a problem there.

HBX001 will pop up when `main()` is not active for longer than 120 seconds. In order for this error to occur, the script had to have been started correctly, and so it is a runtime failure. It is unlikely that there will be a Python exception added to the error log - errors of this nature are either an accidental closing of `main()`, or an unforeseen bug.

FIM001 is a very common (and thankfully simple to correct) error. This occurs when `FileImport.py` is expecting traces to be of `n+1` length, but the traces are something different to that. The fix is very simple: if the traces need to be `N` long, make sure `FileImport` has `if 0 <= int(row[0]) <= N-1:` and `if int(row[0]) == N-1:` in `readFile()`.

FIM002 is a more general error which appears in several instances. Usually, this error code means there was a failure in the analysis somewhere, and that the failure caused a Python exception. This will not cause the code to fail, as it will just bypass that file, but usually this error implies a code design failure. When adjusting `FileImport`, it is important to test it prior to pushing to production.

WDP001 is a WaveDump heartbeat error. If this occurs, the most likely issues are on WaveDump2 as opposed to in the code. Common places to check are if the code is saving to the correct folder (or if it is saving at all), and that WaveDump2 has not crashed.

WDP002 is a full system failure. WaveDump2 has crashed, and the file system has failed as well. Both the code and the files should be checked to see if there was a change made to the software or if the files have been formatted differently.

4.2 Failure Modes

In this section, I will list all known failure modes that didn't get an error code, and explain how to fix them. The errors listed here are generally system errors which require restarting the computer to clear.

Memory leak errors can happen if there are several programs open at once on the production computer. I recommend keeping only the two terminal windows and WaveDump2 open while the scripts are running, and to only open the IDE installed on the computer for debugging purposes while the scripts are down. Leaving the PyCharm IDE open will cause a memory leak and an eventual RAM overload, stopping both scripts and causing the computer to crash.

Encoding failure errors rarely occur when the code writes data points into the memory incorrectly. While this should never occur, it has happened in the past and will result in something like "invalid literal `for int() with base 10: 'Event n. []'`". In this case, take a look at the minutely data set and make sure that all of the data shows as three columns; if it encodes incorrectly, there may just be a single column. This erroneous data is safe to either split into three columns or just delete.

WaveDump2 saving failures can happen when setting up a new run. Sometimes, WaveDump2 needs more than one try to make sure that the files are saving to the correct directory in the host PC. Make sure that it is not saving into the OneDrive and is to the specific local directory on the PC. If the files are still not saving, this can sometimes be fixed by first restarting WaveDump2, setting the file save count quite low to force it to instantiate a temp file, then resetting it back to the corrected value.

WaveDump2 file format can cause its own errors. If the error too many indices `for array: array is 0-dimensional, but 1 were indexed` is returned, there's usually an issue with the way files are going into the indexing. Take a look at the headers on each file, and make sure that they have not changed between prior and current runs. The code has been designed to catch most of these issues, but in the event that it does happen again, it is usually an issue with the importing mechanism.

Flash ADC reading errors can occur when WaveDump2 does not properly read the Flash ADC firmware and drivers. This can occur when moving the program between computers; the solution is to download CAEN's driver, force Windows

to bypass its deprecated driver safety systems, and allow the outdated driver to install. After this, the ADC should be readable by the monitor computer.

4.2.1 Known Limitations

These are not necessarily failure modes of the program, but known limitations of the code that may result in complication if the configuration must be changed.

- Requires CAEN WaveDump2 in ASCII output mode.
- Assumes 10Hz constant sample interval (can be adjusted), and 1-minute sample windows.
- Temporary graph caches reset after each restart.
- Developed and tested on Windows.

5 Maintenance

In this section, I will outline the best practices for keeping the scripts running cleanly, and adjusting them to fit particular needs. Lastly, there are several checklists that succinctly describe how to start and stop the programs.

5.1 Best Practices

There are several important practices to have smooth usage of the monitor and its associated functions in the long-term. I will list them below.

- **Check-ins:** I recommend checking on the monitor computer around once a week. This can be done remotely via Real VNC to the computer. This is important to make sure there is no memory leakage or RAM saturation while the code runs.
- **Regular restarts:** I recommend restarting the computer regularly, around once every two weeks. It can be stretched to once a month or longer if the computer is running fine, but the longer this is stretched, the more likely it is that Windows will fail. Uptime can be checked via the website's status panel, or by looking at the computer's status panel. A good rule of thumb is to just try using the PC - if it is responsive, then it can continue running.
- **Minimum active processes:** It is important for long-term use to keep active processes to a minimum on the computer while detection is running, *especially* the Pycharm IDE or any similarly intense process. Pycharm is known to saturate the detector PC's RAM.
- **Storage clearance:** While the analysis code clears files on its own, it is a good idea to keep an eye on the size of the \Data\Processed folder. Periodically clearing this folder is recommended.

5.2 Adjusting Parameters

The system must be calibrated any time the input voltage to the laser changes. The calibration adjustment is easy to make, but it is important to keep in mind. The input voltage seems to be adjusted depending on the frequency of pulses input from the power supply - it is important to keep a consistent frequency to ensure the data is compatible with previously taken data.

In order to recalibrate the estimated concentrations after the input parameters have been adjusted, one can navigate to Calibration.py and adjust `self.slope = N` to reflect the new calibration constant for whatever their settings are.

The number of samples per trace, and the local position of the peak in each trace must also be standardized. In order to adjust the number of samples in each trace, one must navigate to the class FileReader in FileImport.py. Within this class's readFile function, one must replace N with the correct sample count per trace:

```
1         if 0 <= sample <= N:
2
3         if sample == N:
```

In the same file, if the time delta of the digitizer is adjusted, one can find `self.deltaTime = N` and adjust it there, and anywhere the same time delta is hard-coded (FileImport, ExponentialFit, Grapher).

To adjust the peak position of the traces, one can navigate to the Averager class's align function in SignalProcessor.py, where there is a `target = N` on line 33. This value can simply be replaced with the correct peak position for whatever settings the user would like to use.

If the adjustments on warnings need to be adjusted, for example, to know whether the analysis produced an erroneous value, one can find many of the relevant error messages within app.py and its updater functions. It is effective to search for a particular error code or `eventlogger.log_event` to find all email code lines, and adjustments to the error conditions are directly prior to this line.

5.3 Checklists

5.3.1 WaveDump2 Configuration

- ☐ Start WaveDump2
- ☐ Plug in Flash ADC
- ☐ Open File -> Connect to Device -> Connect to last used device
- ☐ Set Record Length and Pre Trigger position
- ☐ Set Trigger Source to SELF_TRIGGER
- ☐ Edit Mask (hex) to Channel 1
- ☐ Set Trigger Polarity to FALL
- ☐ Scroll down (alt-drag) and set Trigger Threshold and Baseline Level to the correct values, then exit Configuration
- ☐ Make sure Record Length and Pre Trigger position are set correctly
- ☐ Open Output -> Configure
- ☐ Make sure file saving location is set correctly
- ☐ Select One file per Board
- ☐ Set File Header to Yes
- ☐ Set File type to ASCII
- ☐ Check "Split into multiple files"
- ☐ Set file splitting to 600 traces/file (or whatever 1 minute worth of files is)
- ☐ Check Output -> Save Data

5.3.2 Startup

- ☐ Restart the PC
- ☐ Verify Flash ADC is functional and plugged into the USB port on the PC
- ☐ Start WaveDump2
- ☐ Verify WD2 has been configured correctly ([subsubsection 5.3.1](#))
- ☐ Turn file saving OFF in WD2
- ☐ Start WD2 running
- ☐ Start terminal using the .venv inside of \GadoliniumAnalysis
- ☐ Run `python main.py`
- ☐ Enable file saving in WD2
- ☐ Open new terminal using the same .venv
- ☐ Run `python app.py`
- ☐ Monitor the webpage to make sure all systems are functional

5.3.3 Shutdown

- ☐ Stop `app.py` and `main.py` by inputting Ctrl-C in both terminals with nothing selected
- ☐ Press "Stop" in WaveDump2
- ☐ Delete any files in \Data\New